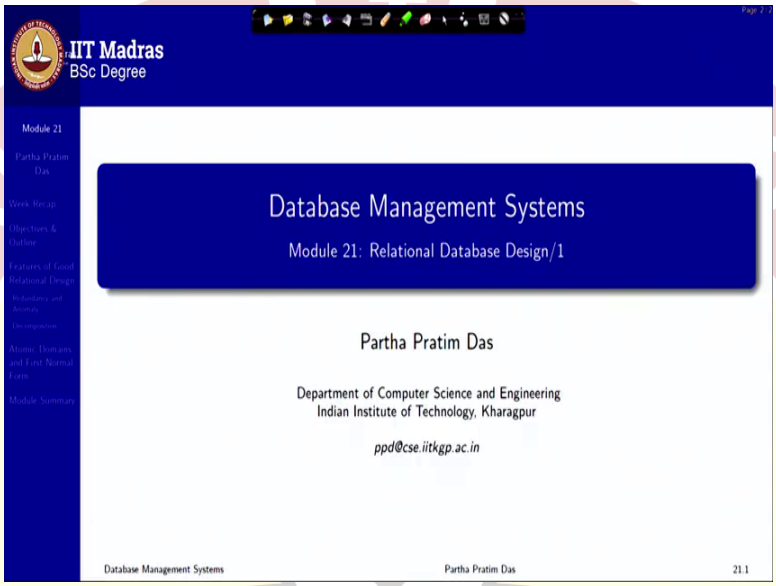
IIT Madras

ONLINE DEGREE

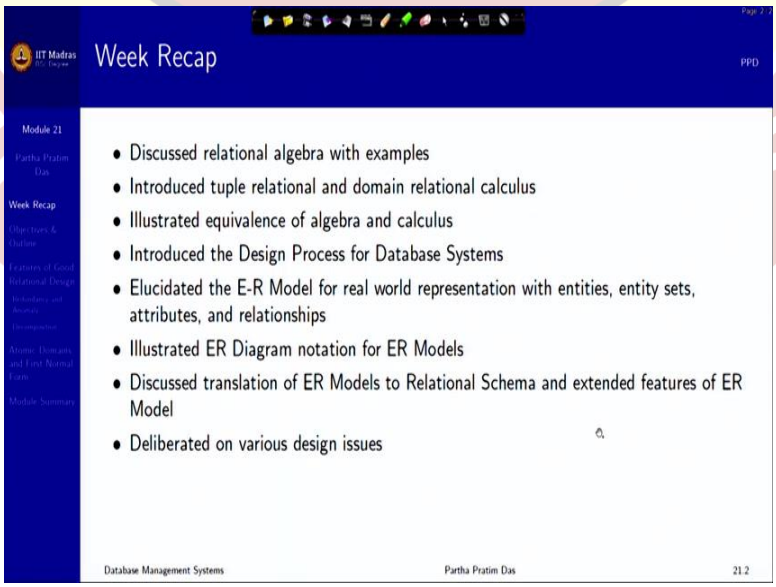
Database Management Systems
Professor Partha Pratim Das
Department of Computer Science & Engineering
Indian Institute of Technology, Kharagpur
Relational Database Design/1

(Refer Slide Time: 00:25)



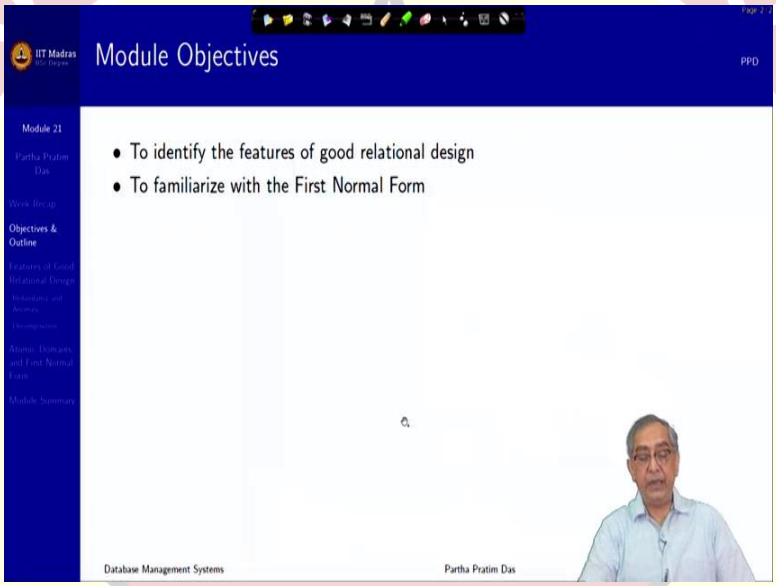
Welcome to week five module 21 of Database Management Systems course in the IIT Madras online B.Sc. program.

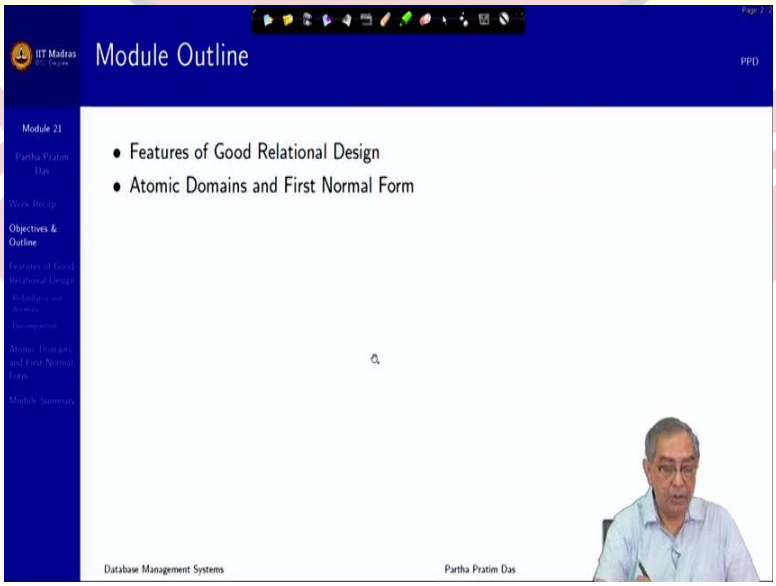
(Refer Slide Time: 00:30)



In the last week, we have dealt extensively with relational algebra, calculus, showing their equivalence and then started off with the design process for database systems. Naturally, the first step in the design is to acquire the requirements and to model them in a compact unambiguous manner for which we have introduced the entity relationship model and the diagrams that are used to represent every entity attribute relationship constraints to a good extent on these requirements. And finally, we discuss the naive translation of this ER models to relational schema and extended features of ER model, deliberated on a few design issues.

(Refer Slide Time: 01:38)



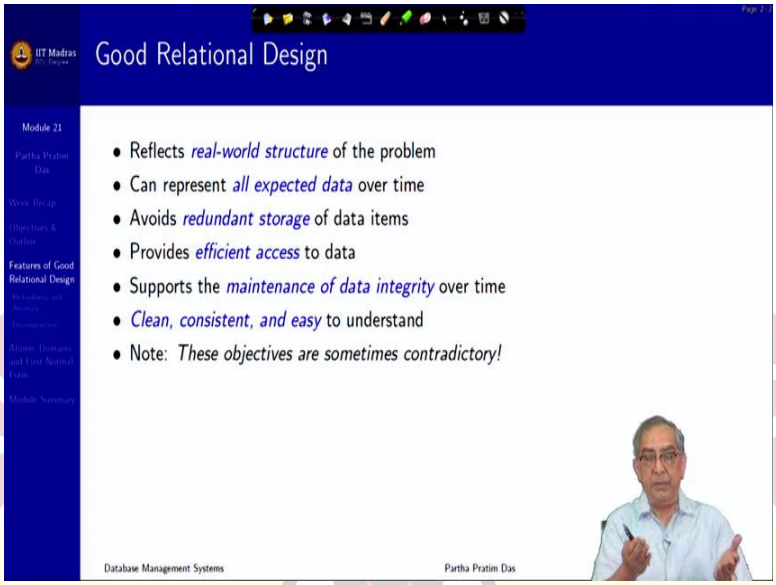


And carrying on with that, we will now start working on the relational database design, which will be a core of this course, a fundamental knowledge to know how to design good databases and how to, what is the theory behind that and how to use that theory to normalize and make the database good. Starting in this module, we will identify the good features of relational design and familiarize you with the first normal form of relations. So, that is what will be covered initially.

(Refer Slide Time: 02:20)

So, first talking about what is a good design, because you can, by now, you must be realizing that given the same set of attributes and properties that we need to keep the data elements there could be multiple ways we can design the tributes, multiple ways we can design the tables.

(Refer Slide Time: 02:24)



So, at a top level here are some, I should say good design practices or good design, appropriate design goals. This is not an exhaustive list, but mostly the strong points we want to talk about. That is your design must reflect the real world structure, because as we explained in terms of modeling, that whatever you will do, you will do based on this design and that must be representative perfectly or adequately of what is happening in the real world or what can happen in the real world or what should happen in the real world.

Then you have to remember that you are doing a design at one point of time. And naturally, the possibilities of data that can come in your database are immense. They are potentially infinite. But your design must be done in a way so that it can represent all expected data over time in future. So, this adequacy of the design is very important.

Third is it should avoid redundant storage of data items, because certainly databases become large, huge. And if you have redundant storage, then you are losing money in terms of the storage. Certainly, it has to be efficient in terms of access. You want the responses to the queries as fast as possible.

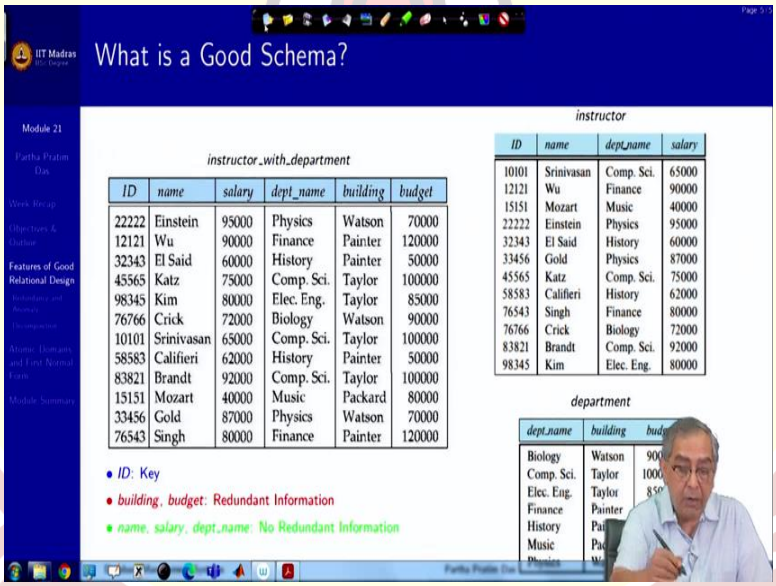
It should support maintenance of data integrity. We have talked about various types of integrities. These must be supportable on your design adequately. And it should be clean, consistent and easy to understand, because certainly, you are not going to live through the entire

lifecycle of the database, others will be there. So, it should be easy for them to understand it should be a clean design.

Now, there could be, I mean, there would be several other goals also, but these are the primary goals. And the fact of the matter is some of these objectives sometimes are contradictory. For example, often as we see is if we want to increase the efficiency of access, then one way could be to use some redundant data to store.

Similarly, there could be, if we have more of redundancy, then certain data integrity issues will become difficult to manage. So, these objectives are interrelated and sometimes contradictory. So, you will have to, I mean, that is always a design problem, how to balance between different factors.

(Refer Slide Time: 05:34)



Now, having said this, let us look at specific examples to somewhat try to understand what is a good design and maybe what is not a good schema. So, you have seen the instructor relation earlier having ID name, department name and salary and the department relation, department name building and budget.

Now, what if, now, this is something which you have seen, which has been done by the designer, but initially what you know. You know that there are instructors, there are departments.

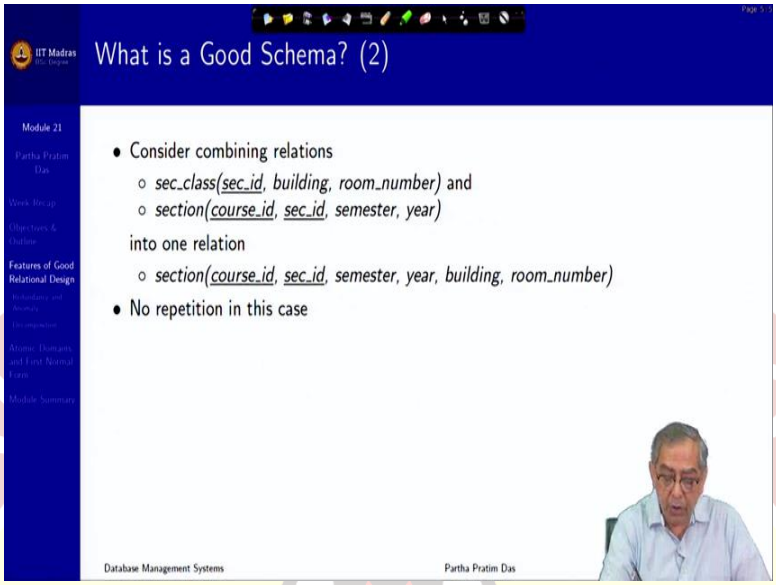
Instructors have certain attributes like salary, the department they belong to, their ID, and departments have buildings and budget.

So, one, if we assume, and that is what often is a point where we start with is suppose I make a table to keep all this information, certainly that will be, that will have adequacy, I mean, that is not a problem. So, if we have that, then we see, well, what are the things we will have to see. First is the key is the ID which is quite obvious, because it has been created for that purpose. Now, if you see, for example, the row of Einstein and Gold, let us look at these two rows, you can see that the building and the budget are repeated. They are duplicates.

Why are they duplicates, because the building and budget is necessarily properties of the physics department and both Einstein and Gold belong to the physics department and it is not limited here. Wherever you have multiple instructors belonging to the same department, the building and budget will be repeated. They are redundant information, because naturally they cannot be different.

At the same time, you can see that things like the name is not a redundant information, the names will be as they are. Salary is not redundant, department name is not redundant. There is no repeatability in this. So, as we see that out of the possible attributes, some in a table could turn out to be redundant, whereas some are not. Now, the question is, how does this impact our design or the issues that we have talked off?

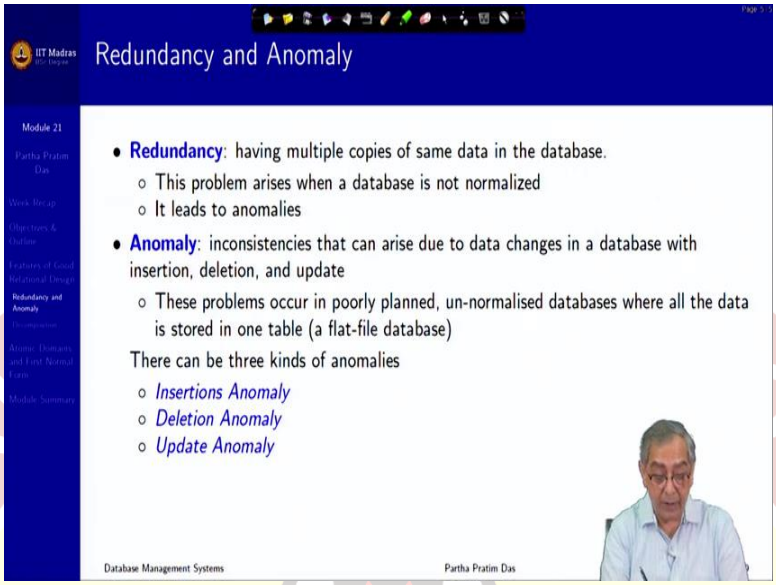
(Refer Slide Time: 08:08)



Now, this is another example where you have the section class. I have marked the keys here and you have section. These are talking about the sections for different courses in a certain semester of a year. These are given as two different schemas. But what if you combine them together, if you put them together? So, depending, I mean you put the section ID attributes, building, room number and also in the section table and make a bigger table. If you start from there, is there a problem.

Now, you can put some thoughts while you study, you will be able to reason that in this case we will not have any repetition. So, what I am trying to point out through these examples is that there are certain cases where we will have repetition and that may also be for certain attributes and then there could be some cases where you do not have repetition. So, this is what we will need to understand to actually make a good design.

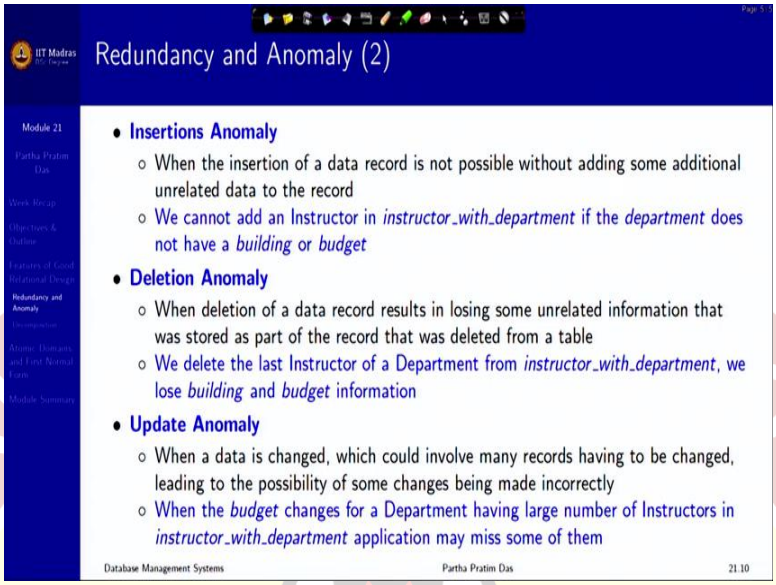
(Refer Slide Time: 09:12)



So, we talk about redundancy. Redundancy is when you have multiple copies of the same data in the database. This happens as we, I mean, we will come to this term normalization. But just to get you familiar with the jargon this happens when the database is not normalized. Now, what is the problem with redundancy? What if we have redundant data? Of course, it will take more storage is a no brainer. But the serious problem is redundancy leads to anomaly. That is inconsistency can arise when you are changing the data in a database that has redundant data.

So, it will happen in all poorly planned databases and there can be three kinds of anomalies; anomalies that happened with insertion, insertion anomaly; anomalies that can happen with deletion, deletion anomaly; and finally, the update anomaly.

(Refer Slide Time: 10:14)



Now, what is an insertion anomaly? Insertion anomaly happens when the insertion of a data record is not possible without adding some unrelated data to the record. For example, let us say, in that whole big instructor with department table, we want to add an instructor who is going to work for a department which does not have a building as of now or which does not have an allocated budget as of now, you will not be able to add that instructor. You know the department, but you do not know department's building or department's budget.

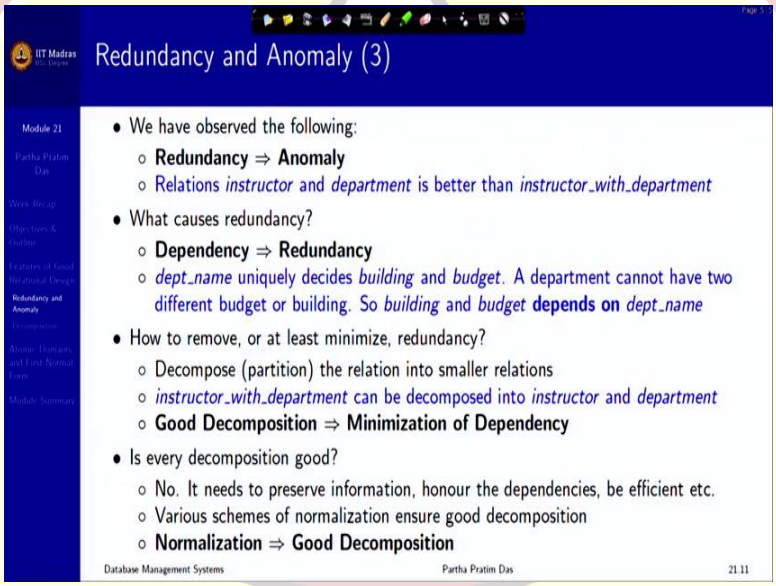
So, you would not be able to add that even though your primary information for addition is an instructor information. If you had instructor and department table separately, you could have added this instructor without any pain, because you know the department name and you do not need to know the department budget of the building. So, this kind of situations is called insertion anomaly.

A deletion anomaly happens when the deletion of a data record results in losing some unrelated information. For example, let us take the, again the case of instructor with department. If you happen to need to delete the last instructor of a department for some reason, then certainly the associated information of budget and building will also get lost, but the department is not disappearing. It is just that currently it is not having any instructor. But when you delete the last instructor since it has all the information together you will also lose the building and budget information. If these two were in two separate tables, this will not happen.

Finally, the update anomaly will happen when you have so much a lot of redundancy, and therefore, the same data is repeated multiple times as we saw in computer science there are three records and so on. Now, suppose in that same case, we want to update the budget, because every year or 6 months the budget gets updated.

Now, this update has to happen in all the three records. So, if your application for some reason of erroneous logic misses out on updating any one of those, then you will have inconsistency. So, having redundancy can cause any one or more of these anomalies and consequent problems to work with.

(Refer Slide Time: 12:54)



So, therefore, we can, we come to the first conclusion that redundancy leads to anomaly. And in our example, the relation instructor and department pair is better than instructor with department big cable. Now, question is what causes redundancy. If you think little bit more, you will find that the redundancy in this case was caused by the fact that department name by itself decides the building and the budget. It is not the instructor or it is not the ID of the instructor or the salary of the instructor. It is the department name which uniquely decides the building and the budget.

So, we say that the building and budget depends on the department name. So, there is a dependency. All attributes are not independent, all attributes are not. As you have come across this idea in terms of discussions on key, but we are kind of generalizing that you can see now that the department name decides building and budget or in other words building and budget

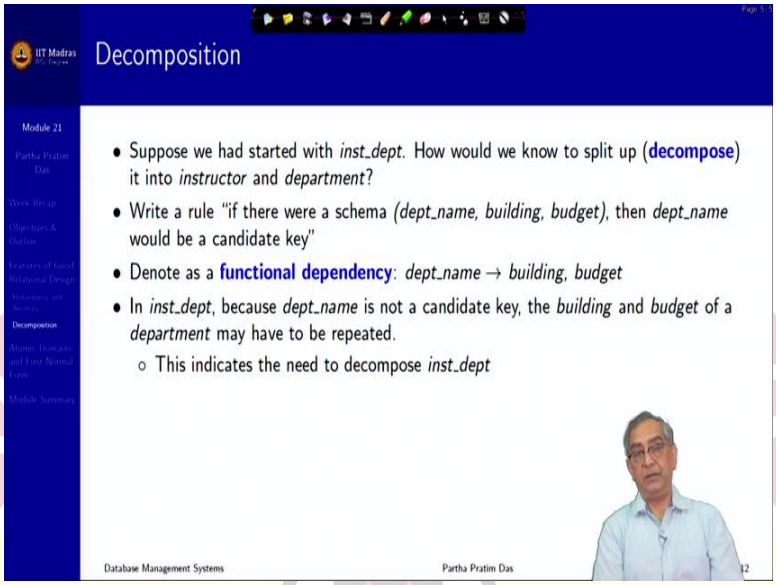
depends on department name, which means that whenever you repeat the department name, if the budget and building are on the same table, they will also get repeated. So, dependency is what causes the redundancy.

So, the question is how to remove or at least minimize the redundancy. The way to do that is to partition the relation, partition the table into smaller tables, smaller relations, as you have seen instructor with department big table. When you partition it in terms of instructor table and department table, then you do not have the redundancy and the consequent anomalies. Now, obviously, this is an example case looks like magic, but there has to be in a structured procedure to do this. So, that is what we are going to learn.

So, good decomposition, we have to know what is the proper way of doing. The good decomposition will mean that dependency will be minimized. Obviously, the consequent question is, is every decomposition good? Obviously, not. It has to have certain properties. It must preserve the information. It must honor the dependencies. It must be efficient. Preserve the information in the sense that after a partition, I should be able to reconstruct the original table.

So, there has to be a way to get that. So, there are various schemes of normalization and which work at different specific cases and gives you good decomposition. So, normalization gives you good decomposition. Good decomposition leads to minimization of dependency. Minimization of dependency leads to minimization of redundancy and that leads to minimization of anomaly. So, we have to normalize. That is the basic lesson that we wanted to.

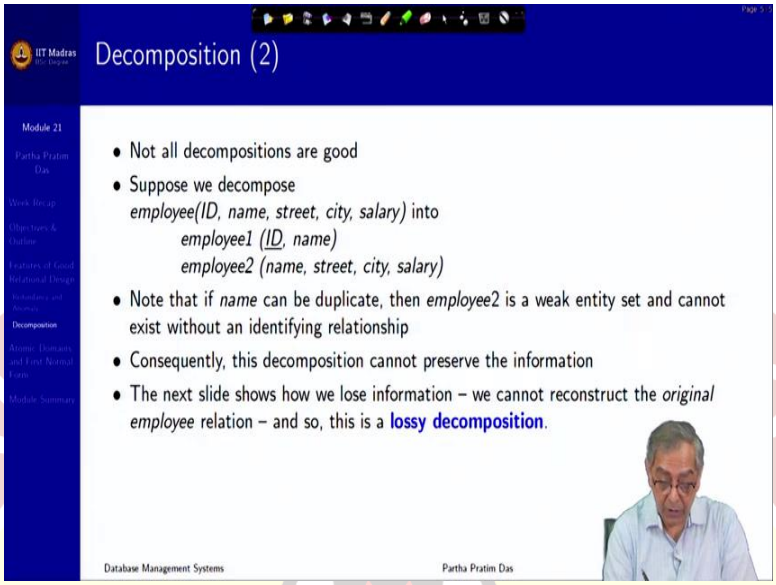
(Refer Slide Time: 16:06)



Now, naturally, we talked about this, but when we say that department name decides building and budget, we formally call it as a functional dependency. So, a functional dependency is like written in the function notation on left hand side you have one or more attributes, on right hand side you have one or more attributes. We say that the left hand side functionally determines the right hand side. That is given a department the building and the budget will be unique. This is the meaning of the functional dependency.

So, the reason we have this, we have had this problem in the instructor with department table is the fact that department decides functionally determines the building and budget, but it is not a key of that relation. We will come to this more. So, we need to decompose based on that.

(Refer Slide Time: 17:09)

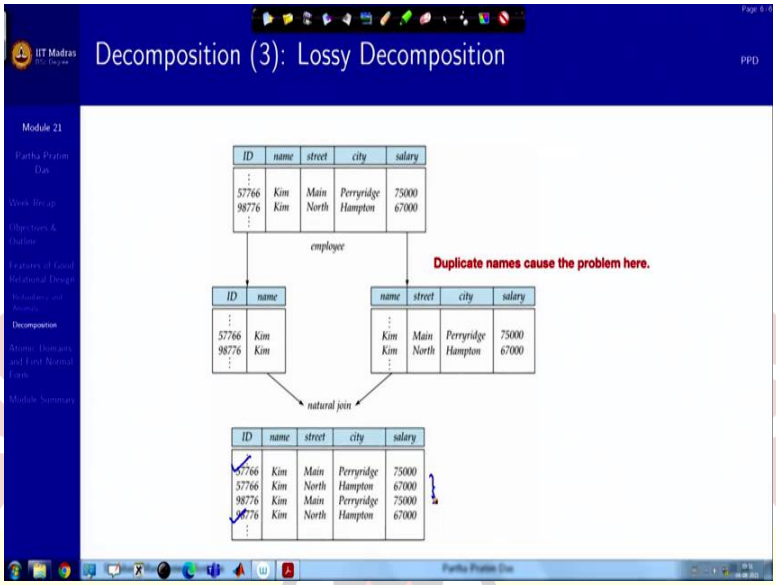


So, let us see what is not a good design. Let us say I have an employee table ID, name, street, city, salary and so on. So, I am going forward and making a decomposition of employee 1 and employee 2 table ID and name and the other is name, street, city, salary. Now, consider employee 2 relation. In employee 2 relation name can be duplicate, two employees can have the same name.

So, which means that employee 2 is a weak entity set. It does not have a key. As we talked about this earlier that it does not have a key, because I can have, I cannot assume that two employees of the same name will not live in the same street of the same city and get the same salary. They are not distinguishable.

So, what did we say that in case of weak entity set how do we model that relation? We have to have an identifying relationship. So, it cannot exist without the identifying relationship. So, let us see what, if we do this kind of decomposition, what kind of problem it can cause, what kind of loss of information it can cause, so which is called lossy decomposition.

(Refer Slide Time: 18:27)



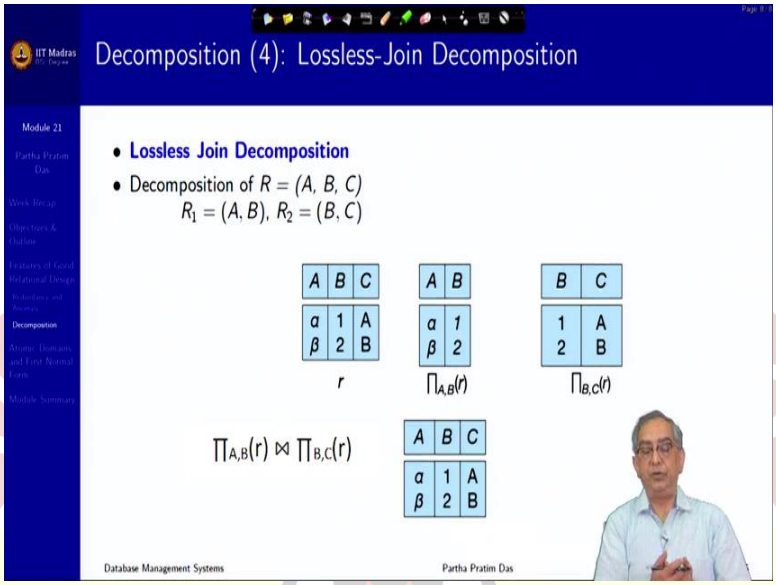
So, there is an instance. Here is an instance of the original employee table together. And we are just showing two records out of that. Now, we have decomposed it employee 1 having ID and name and employee 2 having name, street, city and salary. Now, the question is how do you get the information back, what is common between these two, is a name.

So, note that intentionally we have taken two employees who have the same name. So, how do you get this back? You have a common attribute. How to get the correlated information is to joint. We have talked about natural joint. So, we are doing a natural joint on the common attribute name. And this is what we will get.

So, if you look at then this record is what we had and this record is what we had. I am sorry, just Kim Main Perryridge 75,000, so this we had. Kim, this is Kim, Main or Kim North Hampton this is what he had. So, we had two records, but when they join we have got two more records, additional. Because since name is not unique, when we have done the cross product, we have created entries like this. In the cross product we have created entries like this and like this, which are actually not what is the reality. So, this is called a lossy decomposition.

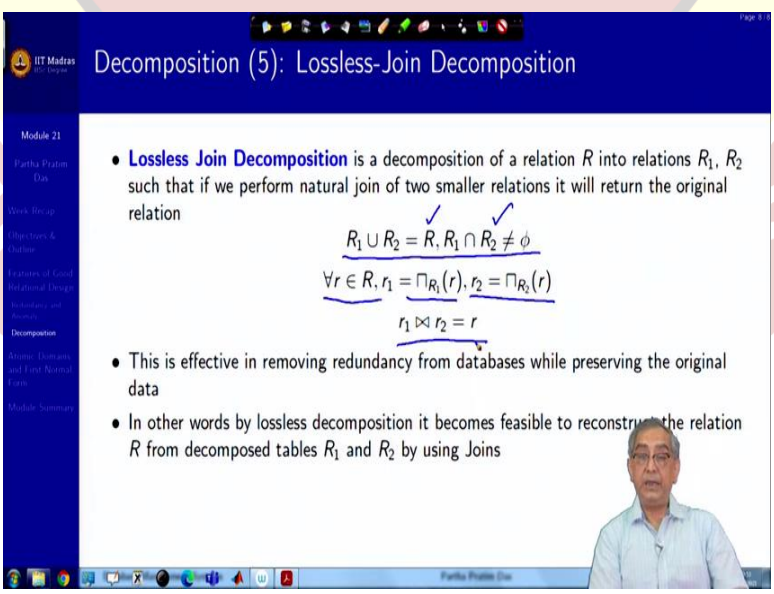
I mean, the sense of loss is different here. I mean, by loss we usually mean less. But if you get more records than what you had started with, then you have lost information as well. So, that is the basic problem and we will have to look for solutions on this.

(Refer Slide Time: 20:42)



So, here is another example, where you can see that we have two, we have a table A, B, C decomposed as AB and BC and we had two records. And if you now do a join, you get back A, B, C, the original table. And this is not just for this instance this has to happen for every possible instance. And then we will say that this is a lossless decomposition or we often say since we get back the information through join we say this is a lossless join information.

(Refer Slide Time: 21:17)

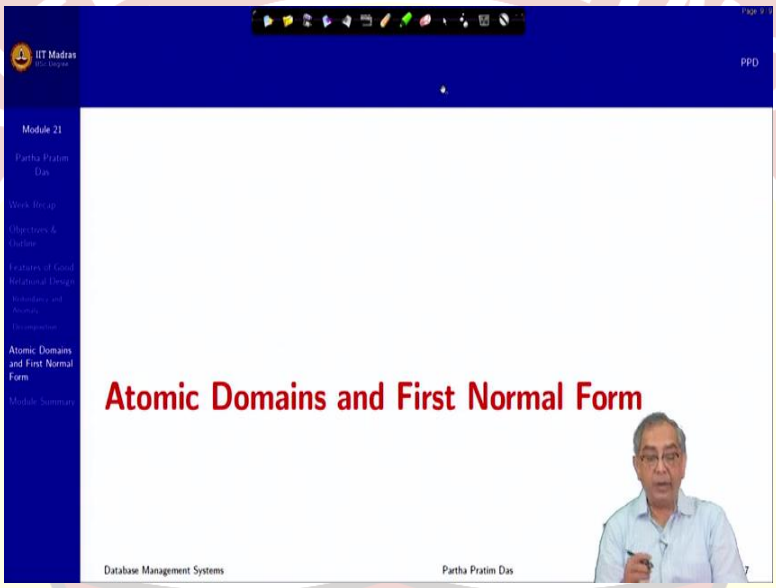


So, in a lossless join decomposition, these are the things that we need, which are very obvious that the two relations must cover all the attributes that is their union must be R. They must have some intersection otherwise you will not be able to join and get back the information. And for every instance of R, if you take

the projection on R1 and projection of R2 those are the instances of the two decomposed relation.

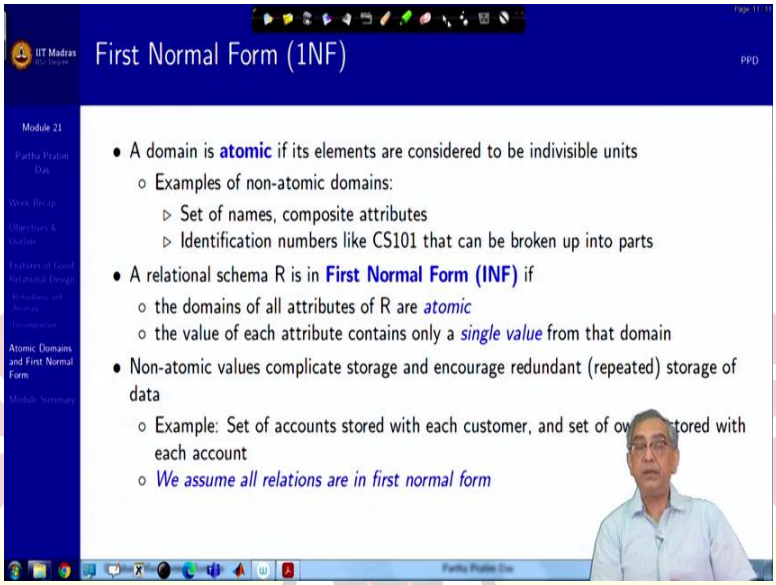
And then if you construct a natural join, you must get back that original relation. And this is, this must happen for all R, that is for all instances of the relation and this is where we are going to see how to ensure that I am just trying to introduce the basic concepts that are involved here.

(Refer Slide Time: 22:10)



Next, what we will talk about is atomic domains and first normal form.

(Refer Slide Time: 22:15)



So, we say, depending on the structure of the relation, we say it is in some normal form or it is no normal form. So, it has different types of first normal form, second normal form, third normal form, fourth normal form, fifth normal form, Boyce-Codd normal form, so on so forth. So, some of these we will study. But to start with the first thing is, what is the first normal form?

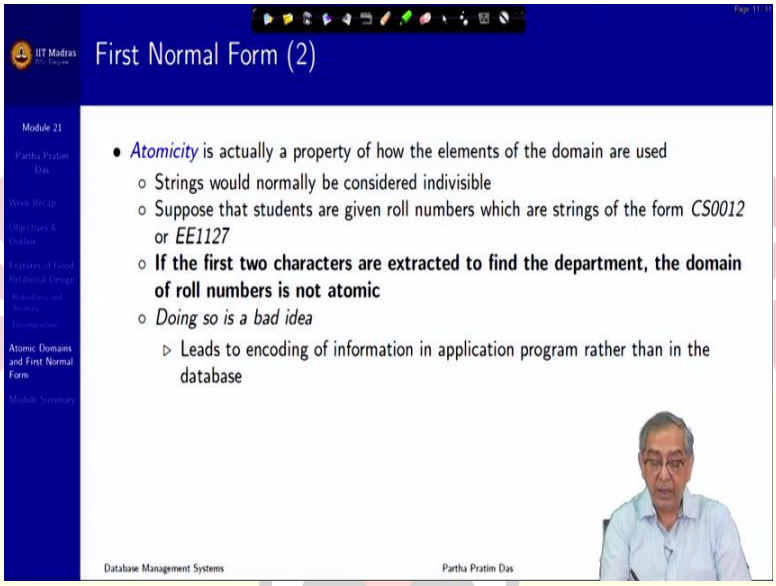
For the first normal form, what we need is we need to deal with atomic domains. What is an atomic domain, where the every, a domain where it every element is considered indivisible. For example, the set of names is a composite domain, because it is a composite attribute, so it is not an atomic domain, because it has first name, middle name, second, third, last name.

Similarly, say identification numbers like CS101 though it is it looks indivisible, but you have encoded in this way so that it can be broken in two parts, so that you can get department, you can get the actual course serial number. So, this will also not be considered atomic. This is a non-atomic domain.

So, we say a relationship, relational schema R is in first normal form, as you say, if domain of all attributes are atomic, every attribute must have an atomic domain. So, you do not have, you should not have name. You should have first name, middle name, last name like that. And every value the attributes can take or every attribute must be single valued, multi-valued attributes are not allowed. You have to do something to take care of those. So, this is a basic premise with

which we start and we assume that all relations are in first normal form. If they are not, then we will have to do something to take them to the first normal form before we can do the next task.

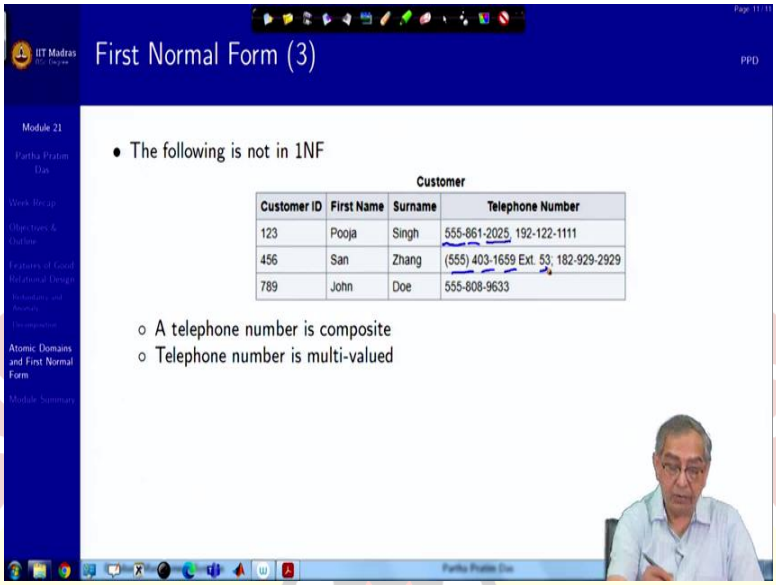
(Refer Slide Time: 24:17)



So, atomicity actually is a property of how elements of the domain are used. For example, strings are always used as strings not as individual characters. So, strings are normally considered indivisible. But as I said, the course number like CS0012 or EE1127 will not be considered atomic, if you intend to extract the first two characters from this course and the domain or say if this is roll number.

So, if you want to extract the first two characters to get the department and then the actual number, so your roll number is not an atomic entity, because how do you extract a part and deal with it, which means that you are not remaining within the database. You are going to the application program which will take out the first two characters, find what is the department associated information so this leads to the information being managed by the application program and not within the database. So, that causes the potential difficulties and therefore, will be avoided.

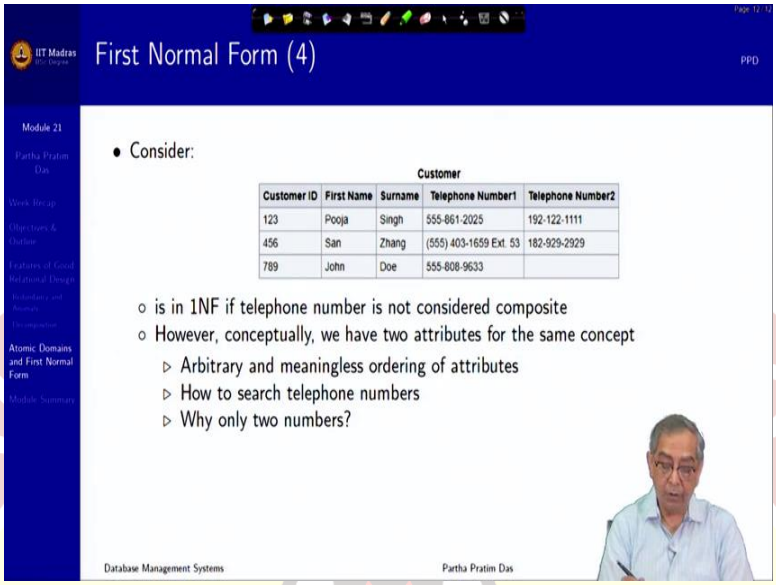
(Refer Slide Time: 25:37)



Now, let us have a relation which is not in first normal form. So, this is one. You have customers, their name, surname and you have telephone numbers. Now, there are two issues. First of all, this is not in first normal form for two reasons; one is telephone numbers are not indivisible, are not atomic, they are like this is, this basically is written in the U.S. style. We probably will write in a different style. But the fact is that every country's telephone numbers have certain components.

So, you can see that again like the roll number case here you have three components here, you have three components here, actually you have a four here and so on. So, telephone number is composite, therefore, this is not in 1NF number. Further, a customer may have multiple telephone numbers. So, if the customer has multiple telephone numbers, then naturally it is multi-valued. And therefore, by that condition also this is not a 1NF schema.

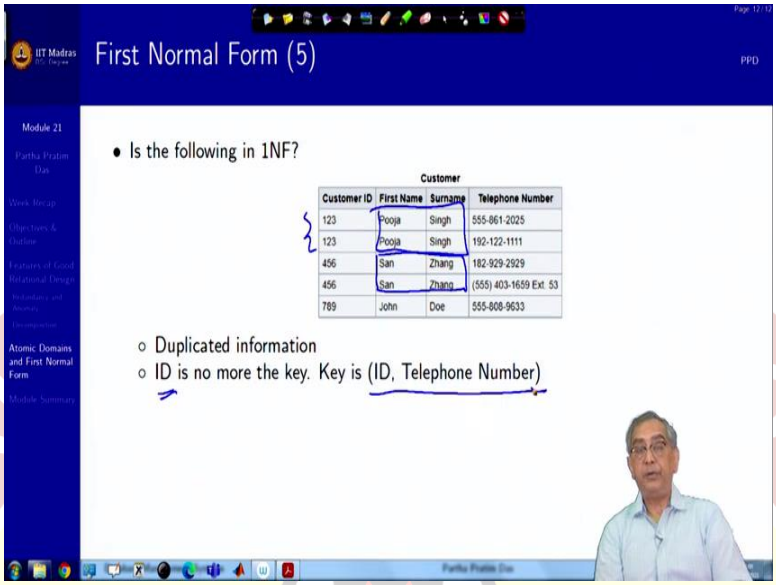
(Refer Slide Time: 26:50)



So, what I do is, we will say that we will have two telephone number fields and put them together. So, we now have two telephone number fields each having one number each. Now, first, let us assume that the telephone number is not composite, let us say it is just, altogether is just a, here it is a 10 digit or 12 digit, whatever number if we just assume that. So, but what is a problem still. There are several problems. We have tried to create two attributes to take care of the case of multiple values.

First of all, these two attributes conceptually mean the same things, but we have just given them two. So, when you query, you would not know which attribute to query on. So, how do you search the telephone number? Should you use telephone number one or telephone number two or both, all these. Then the question is, obviously, why two numbers, somebody may have five numbers. This would not represent that. So, this kind of, using multiple attributes to make it into first normal form is not going to work.

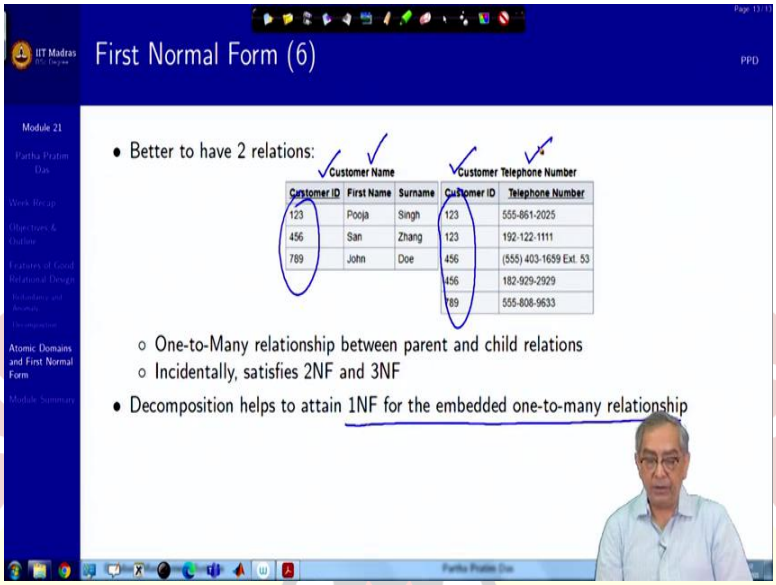
(Refer Slide Time: 28:12)



So, what we do is now we have one field only and we use separate entries for separate telephone numbers of the customer. So now what you have since Pooja Singh has two telephone numbers, so you have two records for this. Obviously, the first problem is you have redundancy, you have duplicate information, redundancy coming in, which is as we saw is bad.

The second issue that comes that earlier ID was the key, the customer ID was the key, but now it cannot be, because there are multiple records with the same ID. So, in a very clear way ID and telephone number pair becomes the key. How do you want to deal with because a telephone number is a frequently changing number, a number can get bad, it may someone can discontinue or add a new number and so on, so forth.

(Refer Slide Time: 29:25)

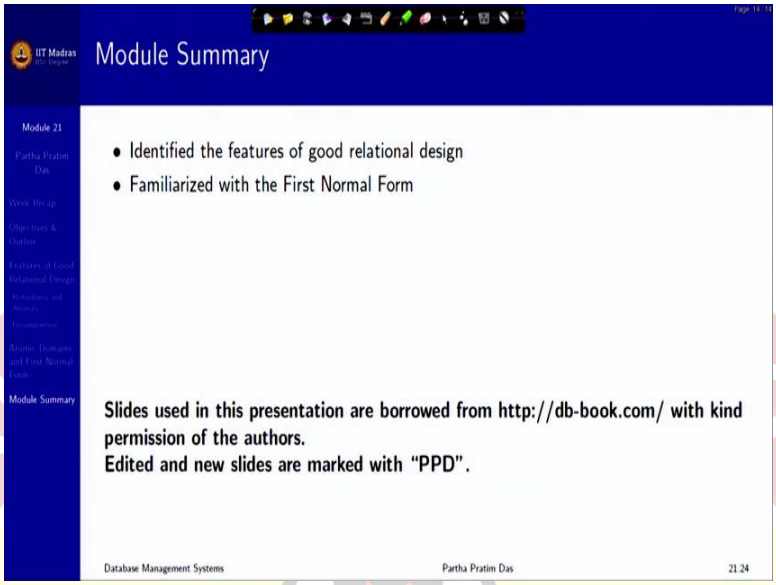


So, what is a better solution? A better solution is you can see that these are two different concepts, one is the customer its name and so on, and one is a telephone numbers. So, why not, so separate it out. Do not keep them together. Have a customer table which has the customer information and have a customer telephone number table which has a telephone number information. So, the customer ID is a common attribute through which you will be able to join and get the original information back.

So, it is kind of, why is this happening, because there is a one to many relationship between the parent concept and the child concept or between customer ID and the telephone number. One customer ID can have multiple telephone numbers. That is, I mean, this is the key observation. Rest of it is an example. But whenever you have this, when you have within a relation a one to many relationship between attributes, you will, you are going to have this problem.

So, the solution to that is to decompose so that you have, now this is in 1NF, this also is in 1NF assuming, of course, that the telephone numbers are indivisible, they are not a composite attributes. So, this is how your decomposition can really start making the cleanup of the data and it starts with attending the first normal form for all relations as you need.

(Refer Slide Time: 31:08)



So, in this module, we have identified the good, some good features of the relational design and discuss the basic issue of why you need normalization to reduce redundancy and anomaly and we are familiarized with the atomic domains and first normal form. Thank you very much for your attention and we will meet in the next module.